



VERITRUSTLAB

LEARNING AND CERTIFICATION SYSTEM

Production Implementation Plan

Learning UX | Assessment | Verifiable Credentials

Purpose

Add a scalable, secure and deeply interactive learning domain to the existing VeriTrust workspace, from discovery through verifiable certification.

Prepared for VeriTrust | July 2026 | Version 1.0

Contents

A complete current-state analysis and production blueprint for learning, practical labs, assessments, verifiable certificates, engagement, security, data, APIs, operations and delivery.

1. Executive decision and target outcome	4
2. Current project architecture and integration constraints	4
3. Navigation audit outcome and target information architecture	5
4. Product principles and learning experience	6
5. Launch curriculum and content operating model	7
5.1 Recommended launch certification	7
5.2 Future paths	7
5.3 Content lifecycle	7
6. Learner journey and interaction design	8
6.1 Discovery and onboarding	8
6.2 My Learning	8
6.3 Lesson player	8
6.4 Feedback and recovery	9
7. Engagement system that produces meaningful interaction	9
8. Assessment architecture	10
8.1 Assessment types	10
8.2 Blueprint and scoring	10
8.3 Attempt state machine	10
8.4 Integrity controls	11
8.5 Item quality	11
9. Certification and credential lifecycle	11
9.1 Eligibility and issuance	11
9.2 Credential contents	11
9.3 Public verification	12
9.4 Expiry, renewal, and revocation	12
10. Data model and database constraints	12
11. API design and Vercel integration	13
11.1 Learner and public routes	14
11.2 Organization and content routes	14
11.3 API rules	14

12. Frontend component and state plan	15
13. Product labs and VeriTrust integration	15
14. Entitlements, commerce, and organization features	16
15. Security, privacy, and abuse prevention	16
16. Accessibility, localization, and inclusive assessment	17
17. Analytics, experimentation, and reporting	17
18. Reliability, performance, and observability	18
19. Testing and quality strategy	18
19.1 Automated layers	18
19.2 Certification release gates	19
20. Delivery roadmap, dependencies, and team shape	19
21. Prioritized implementation backlog	20
P0 - required for a trustworthy launch	20
P1 - high-value engagement and organization readiness	20
P2 - scale and advanced credentials	20
22. Definition of done and launch scorecard	21
Functional	21
Security and integrity	21
Experience and inclusion	21
Operations and outcomes	21

Document version: 1.0 Planning date: July 2026 Status: Implementation blueprint Audience: Product, engineering, content, security, design, QA, operations, and leadership

1. Executive decision and target outcome

VeriTrust should add a first-class Learning and Certification domain to the existing authenticated workspace, not a collection of disconnected static course pages. The target system must help a learner discover a relevant path, complete short lessons and practical security labs, prove competence through controlled assessments, receive a verifiable certificate, and return for reinforcement or recertification. It must also give organization administrators a reliable view of learner progress without exposing private answers or weakening assessment integrity.

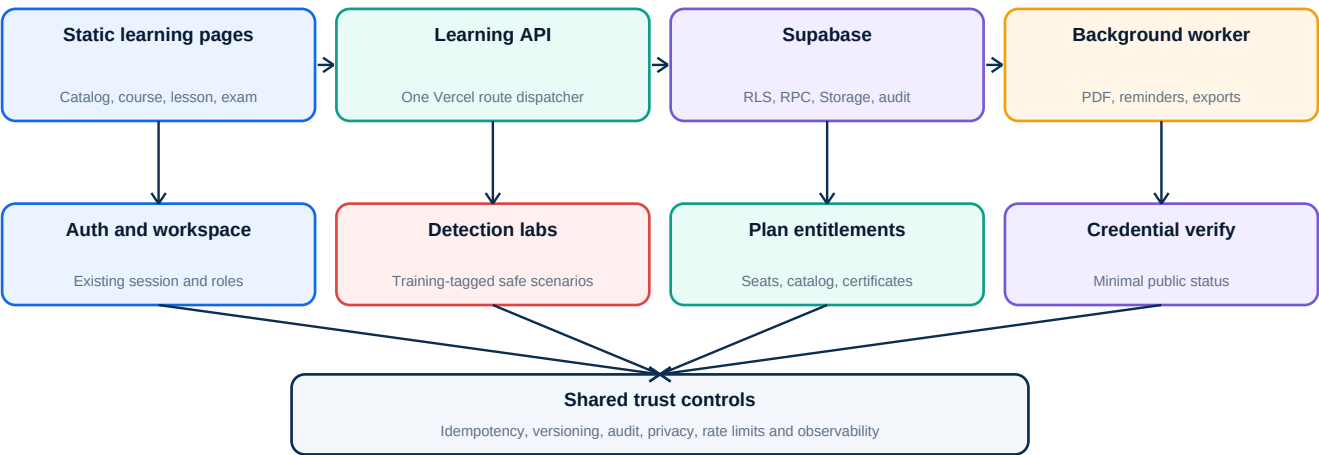
The recommended delivery is a modular learning service implemented within the current static HTML, browser JavaScript, Vercel Functions, Supabase, and Stripe architecture. A new consolidated `api/learning.js` function should own learning commands and queries so the repository stays within the Vercel function budget. Supabase should remain the system of record for catalogs, versioned content metadata, enrollments, progress, attempts, credentials, audit events, and row-level authorization. Supabase Storage should hold media, downloadable resources, and generated certificate artifacts.

The first release should contain one excellent certification path rather than many shallow courses. The recommended launch path is Digital Trust Analyst - Foundation, built from VeriTrust's existing product strengths: suspicious-message triage, URL risk, synthetic-media awareness, safe interpretation of model output, human review, and responsible reporting. It should combine concise instruction, interactive checks, authentic examples, product-assisted labs, a practice assessment, and a controlled final exam.

The engagement goal must be measurable and ethical. No implementation can guarantee 100 percent interaction or completion. The system can maximize meaningful interaction through immediate relevance, visible progress, useful feedback, short learning loops, accessible design, practical labs, spaced reinforcement, and responsible reminders. Success must be judged with activation, lesson completion, mastery, certification, retention, satisfaction, and accessibility metrics rather than raw clicks.

2. Current project architecture and integration constraints

Figure 1. Learning domain integrated into the current VeriTrust platform



One additional consolidated function preserves the existing Vercel, Supabase, billing and worker patterns.

The repository is a multi-page web platform with 20 root HTML pages, shared CSS and browser JavaScript, consolidated Vercel serverless APIs, Supabase authentication and persistence, Stripe billing, AI detection routes, developer API keys, and a durable gateway worker subsystem.

Current area	Observed implementation	Learning-system implication
Frontend	Static HTML pages with shared <code>assets/css/veritrust.css</code> , page-specific CSS, and browser JavaScript	Add reusable learning page shells and shared <code>learning.js</code> ; do not introduce a second visual system

Current area	Observed implementation	Learning-system implication
Navigation	Runtime normalization in <code>assets/js/site.js</code> , workspace subnavigation, auth-aware links	Add Learn to the centralized primary navigation and Learning to the workspace navigation contract
Authentication	Supabase Auth REST calls; browser session stored locally; protected pages redirect to <code>auth.html?redirect=...</code>	Reuse the same session and safe-return behavior for enrollment, lessons, exams, and certificates
Organization model	Server context exposes user, profile, organization, membership role, plan, and usage	Scope assignments, cohorts, admin views, and entitlements to organization membership
API shape	Six consolidated Vercel Functions dispatch to route handlers	Add one consolidated learning function and route handlers; avoid a function per endpoint
Persistence	Supabase REST/RPC access with service-role operations and compatibility fallbacks	Use migrations, RLS, RPC transactions, and immutable version records; avoid compatibility fallbacks for assessment integrity
Billing	Plans, organization subscriptions, Stripe checkout, portal, webhook sync, entitlement snapshots	Add plan features such as learning access, seats, certificates, and admin analytics through the same entitlement layer
Detection	Deepfake, phishing, and link checks persist scans and normalized results	Reuse selected scans as lab evidence while separating learning completion from model verdict correctness
Gateway	Policy, evidence, review, worker, retention, webhook, and audit patterns	Reuse idempotency, audit, versioning, and state-machine practices for exams and credential issuance
Testing	JavaScript syntax, Vercel function-budget checks, a small Node test suite	Add schema, authorization, calculation, navigation, accessibility, API contract, and end-to-end tests

Important prerequisite: the README references `docs/supabase-production-schema.sql`, but that schema or an equivalent tracked migration directory is not present in the reviewed project files. The learning module must not begin with ad hoc dashboard SQL. Establish version-controlled migrations first, capture the deployed baseline, and add a CI migration check. This is a release blocker because assessment and certificate integrity depend on reproducible database constraints and policies.

The project also currently treats all pages except the landing and authentication pages as protected in `assets/js/site.js`. The plan assumes the catalog summary can be public if product leadership wants search visibility, while enrollment, lessons, progress, exams, and personal certificates remain authenticated. This should become an explicit route-access matrix rather than an implicit two-page allow-list.

3. Navigation audit outcome and target information architecture

The navigation audit checked all internal page targets and document fragments. One intent mismatch was found and corrected: the landing CTA labeled Start Free Scan referenced the dashboard instead of the detection chooser. The password-reset action also used a placeholder `#` anchor; it is now a semantic button. A navigation integrity script now checks all 20 pages, internal fragments, and critical CTA contracts during `npm run check`.

The learning launch should extend that same contract rather than hand-editing divergent menus on every page.

Entry point	Target	Authentication behavior	Primary user intent
Global nav: Learn	<code>/learn</code>	Public summary or authenticated catalog, by product decision	Discover available learning paths
Landing CTA: Learn Digital Trust	<code>/learn</code>	Preserve destination through sign-in	Understand value before enrollment
Dashboard card: Continue Learning	<code>/learn/my-learning</code>	Required	Resume the next incomplete activity
Workspace nav: Learning	<code>/learn/my-learning</code>	Required	View enrollments, progress, deadlines, and credentials

Entry point	Target	Authentication behavior	Primary user intent
Catalog card	/learn/courses/:slug	Public preview; sign-in to enroll	Evaluate outcomes, level, time, and prerequisites
Enroll / Start	/learn/courses/:slug/start	Required; redirect back after login	Create or resume enrollment
Lesson next action	canonical next lesson or assessment	Required	Continue without returning to the catalog
Certificate card	/certificates/:public-code	Owner view includes downloads; public view exposes minimal verification	Share or verify a credential
Admin learning	/learning-admin	Organization owner/admin/instructor only	Assign paths and monitor aggregate progress

Recommended page shells and clean routes:

- [learning.html](#) -> </learn>, </learn/catalog>, </learn/my-learning>
- [course.html](#) -> </learn/courses/:slug>
- [lesson.html](#) -> </learn/courses/:slug/lessons/:lessonSlug>
- [assessment.html](#) -> </learn/assessments/:assessmentId>
- [certificate.html](#) -> </certificates/:publicCode>
- [learning-admin.html](#) -> </learning-admin>

Vercel rewrites should route these paths to static shells while JavaScript reads the route parameters and calls authenticated APIs. Every route must support direct opening, refresh, browser back/forward, canonical titles, loading states, empty states, expired-session recovery, and safe error recovery. The global navigation source in [site.js](#) must be the single primary-menu definition. Workspace links should be generated from one configuration or a shared include/build step so page-to-page references cannot drift.

4. Product principles and learning experience

1. Outcome before content. Every course begins with a concrete capability statement and a short diagnostic or scenario, not a long introduction.
2. Practice before proof. Learners must receive guided practice and feedback before a scored certification attempt.
3. Short loops. Target 5 to 9 minutes per lesson and 20 to 35 minutes per module, with resumable state after every meaningful interaction.
4. Authentic work. Labs use suspicious messages, URLs, media examples, report interpretation, and safe escalation decisions similar to real VeriTrust workflows.
5. Transparent AI. Learners are taught that model output is decision support, not proof. No certificate should imply forensic authority that the product cannot support.
6. Mastery, not attendance. Completion time alone never grants a certificate. Required objectives must be measured by assessment blueprints.
7. Ethical engagement. Progress, reminders, streaks, and rewards must help the learner reach a stated goal. No shame language, artificial urgency, hidden opt-ins, or misleading scarcity.
8. Accessible by default. Keyboard, screen-reader, reduced-motion, caption, transcript, contrast, zoom, and mobile requirements are acceptance criteria.
9. Version everything that affects a credential. Course version, exam blueprint, question revision, scoring rule, certificate template, issuer, and policy must be reconstructable.
10. Minimize personal data. Collect only what is required for identity, progress, credential display, audit, and organization reporting.

5. Launch curriculum and content operating model

5.1 Recommended launch certification

Digital Trust Analyst - Foundation

Module	Learning outcomes	Interactive work	Evidence of mastery
1. Digital trust foundations	Distinguish triage, evidence, confidence, risk, and proof	Classify claims as safe, overstated, or unsupported	Scenario checks
2. Phishing and social engineering	Identify urgency, credential, payment, identity, and channel indicators	Annotate suspicious messages; compare model and rule signals	Case decisions and rationale
3. Link intelligence	Interpret domain, subdomain, encoding, redirect, impersonation, and HTTPS signals	Decompose URLs and run safe string-only checks	URL analysis lab
4. Synthetic media awareness	Explain deepfake model limitations and safe review steps	Compare normalized scores and evidence statements	Result-interpretation lab
5. VeriTrust workflows	Run web checks, interpret reports, save evidence, and escalate responsibly	Guided product labs using seeded safe examples	Lab completion events
6. Responsible decisions	Apply human review, privacy, and incident-handling principles	Branching incident simulation	Final scenario score
Certification exam	Demonstrate integrated foundation competence	Blueprinted randomized assessment	Passing scaled score plus objective floors

Target learning time is 4 to 6 hours, divided into six modules, 24 to 32 lessons, four labs, one practice assessment, and one final exam. Course preview must show level, estimated time, prerequisites, objectives, version, certificate validity, exam policy, and accessibility features before enrollment.

5.2 Future paths

- Phishing Triage Specialist
- Link Intelligence Analyst
- Synthetic Media Review Fundamentals
- VeriTrust API Practitioner
- Unified Security Gateway Operator
- Organization Security Awareness track for non-technical employees

Each path should reuse competency objects and lesson blocks where appropriate but publish an immutable course version. Reuse must not create a hidden dependency where editing one shared lesson changes a course already tied to issued certificates.

5.3 Content lifecycle

Content state: draft -> editorial review -> security review -> accessibility review -> approved -> scheduled -> published -> superseded -> archived.

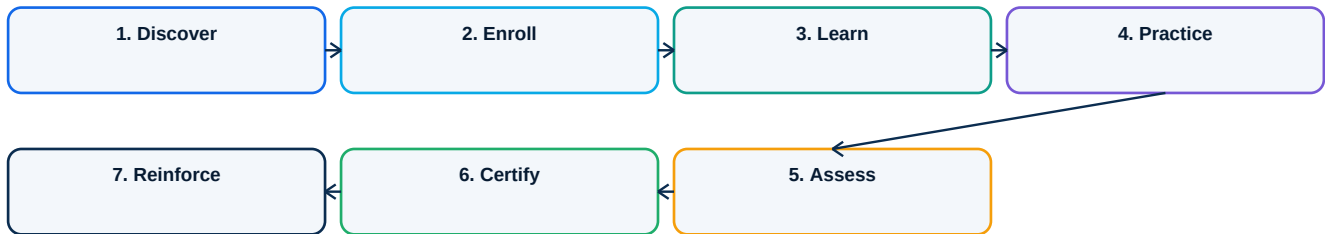
Only a published course version can accept production enrollments. Publishing creates a content manifest hash and freezes objective mappings, required activities, assessment blueprint, completion rules, and certificate policy. Corrections that affect meaning create a new version. Cosmetic corrections can be recorded as a non-material revision with an audit note.

Required roles:

- Author creates content and question drafts.
- Subject-matter reviewer verifies technical claims and limitations.
- Assessment reviewer validates objectives, difficulty, distractors, and fairness.
- Accessibility reviewer checks semantics, alternatives, captions, reading order, and interaction.
- Publisher approves the immutable version.

6. Learner journey and interaction design

Figure 2. Learner interaction loop



Every stage has a clear next action, feedback signal and resumable server state.

6.1 Discovery and onboarding

The catalog should start with a goal selector: Protect myself, Review suspicious content, Use VeriTrust at work, Integrate the API, or Manage a security program. It then recommends one path with a clear explanation. Filters include level, time, topic, language, certificate, and accessibility format. Search must match title, outcomes, tags, and competencies.

On first enrollment, ask at most three useful questions: role or goal, experience level, and weekly learning target. All are editable and skippable. A 5-minute diagnostic can recommend skipped practice, but it must not waive required final-assessment objectives unless an explicit prior-learning policy supports that.

6.2 My Learning

The authenticated home shows:

- one Continue button resolved server-side to the best next activity;
- progress by course and objective, not only a percentage;
- estimated remaining time based on required activities;
- next deadline or certificate-expiry date;
- recent achievements and certificates;
- review queue for spaced practice;
- organization assignments separated from self-enrollments;
- offline/downloadable resources where permitted.

The learner should never need to remember where they stopped. Progress writes occur after meaningful events, the next activity is deterministic, and opening a second device reconciles with the latest server version.

6.3 Lesson player

The lesson page uses a stable three-region layout: course outline, lesson content, and progress/action bar. On mobile, the outline becomes a drawer and the action bar remains reachable without obscuring content.

Supported block types:

- rich text with callouts, definitions, and code;
- image with meaningful alternative text;
- video with captions, transcript, playback speed, and chapter markers;
- knowledge check with immediate explanatory feedback;
- card sort, match, hotspot, ordering, or decision matrix with keyboard equivalents;
- branching scenario with consequences and rationale;
- product lab launch and evidence capture;
- downloadable checklist or reference;

- reflection prompt stored privately unless the learner explicitly shares it;
- end-of-lesson summary and confidence check.

Completion is event-based. Reading blocks can require a minimum visible engagement threshold, but elapsed time is never the sole signal. Interactive blocks require a valid submitted state. Video completion accepts transcript-based equivalence and accessible alternatives. Learners can revisit anything without losing completion.

6.4 Feedback and recovery

Practice feedback explains why an answer is strong or weak, links to the objective, and offers one targeted retry. Repeated failure changes the help level: hint, worked example, prerequisite lesson, then optional instructor support. The interface never labels the learner as bad or behind.

When a network write fails, the UI stores a bounded pending event queue, shows Saved locally, retries with idempotency keys, and confirms when synced. Exam responses use stricter server acknowledgement and visible connection status.

7. Engagement system that produces meaningful interaction

Engagement should be designed as a learning loop:

Goal -> relevant activity -> active response -> immediate feedback -> visible mastery -> next best action -> spaced return.

Recommended mechanisms:

Mechanism	Learner value	Guardrail	Measurement
Progress map	Makes the route and remaining work clear	Never inflate completion	Course and objective completion
Micro-lessons	Reduces startup cost	Do not fragment coherent concepts	Start-to-complete ratio
Practical labs	Connects learning to VeriTrust usage	Use seeded non-sensitive examples by default	Lab launch and successful completion
Adaptive review	Revisits weak objectives	Explain recommendations; allow dismissal	Improvement on repeated objective checks
Streak	Supports a self-selected habit	Use weekly consistency; allow pause; no loss framing	Active learning days, not page visits
Milestones	Recognizes real progress	Award only for defined achievements	Milestone-to-next-lesson conversion
Reminders	Helps learners return to a chosen goal	Opt-in, frequency controls, quiet hours, unsubscribe	Helpful return rate and unsubscribe rate
Peer or cohort prompts	Creates accountability	Organization-controlled, moderated, privacy-safe	Cohort completion and participation
Certificate preview	Clarifies the outcome	Never imply a pass before assessment	Enrollment and exam readiness

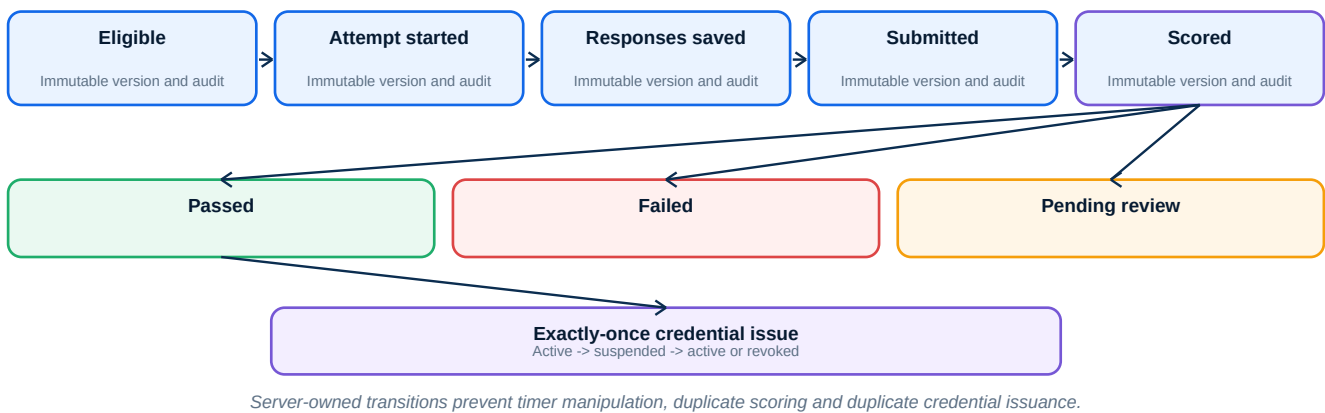
Initial product targets should be treated as hypotheses, not promises:

- 70 percent of enrolled learners start the first required activity within 24 hours.
- 80 percent of started micro-lessons reach completion.
- 55 percent of activated self-paced learners complete the launch course within 30 days.
- 75 percent of practice-assessment users take a next recommended action.
- 85 percent of issued certificates are successfully retrievable and verifiable without support.
- Accessibility task completion remains within 10 percent of the overall completion rate.

Use cohort analysis and controlled experiments. Never optimize for click volume if knowledge gain, satisfaction, or accessibility declines. A feature ships broadly only if it improves at least one learning outcome without materially harming another guardrail metric.

8. Assessment architecture

Figure 3. Assessment and credential state controls



8.1 Assessment types

- Knowledge checks: unlimited, immediate feedback, ungraded or low stakes.
- Module checks: blueprint-aligned, multiple attempts, targeted remediation.
- Practice exam: exam-like timing and coverage, but separate items and full feedback after submission.
- Certification exam: controlled attempt, randomized forms, delayed answer disclosure, audit trail, and credential eligibility.
- Practical lab assessment: deterministic checks against seeded tasks or structured reviewer rubric.

8.2 Blueprint and scoring

Every assessment version has an explicit blueprint that maps objectives to item counts, cognitive level, format, difficulty range, and criticality. The exam service creates an immutable attempt form from the published blueprint and item revisions.

Recommended launch scoring:

- 40 to 55 scored items plus up to 5 unscored calibration items;
- 60 to 75 minutes;
- scaled passing score equivalent to approximately 75 percent, calibrated after pilot data;
- minimum 60 percent in each critical objective group so strength in one topic cannot hide a dangerous gap;
- final integrated scenario weighted more heavily than simple recall;
- no negative marking;
- two attempts per 30 days, with targeted remediation required after failure;
- result status: pass, fail, pending review, invalidated, or interrupted.

The score calculation must run on the server inside one transactional submit operation. The client never receives correct answers for certification items. Item order and option order are randomized from a stored seed. Changing a question creates a new revision; attempts retain the exact revision IDs.

8.3 Attempt state machine

created -> identity_ready -> in_progress -> submitted -> scoring -> passed | failed | pending_review

Exceptional terminal states: expired, abandoned, interrupted, invalidated.

Only legal state transitions are accepted. Starting and submitting are idempotent. The server owns `started_at`, `expires_at`, `submitted_at`, and remaining-time calculation. Client clocks are display aids only. Each response write includes attempt ID, item instance ID, response revision, and idempotency key.

8.4 Integrity controls

Use risk-tiered integrity rather than intrusive surveillance by default.

Tier	Use	Controls
Practice	Learning and readiness	No restrictions; full feedback after submission
Standard certification	Individual foundation credential	Signed-in identity, attempt limits, randomized form, server timer, response audit, anomaly signals, item exposure controls
Verified organization	Employer-required credential	Organization assignment, verified name, optional reviewer approval, stricter environment checks
Proctored future tier	High-value external credential	Separate consent, approved provider, regional privacy review, accommodations, appeal process

Anomaly signals can include impossible response speed, identical uncommon response patterns, excessive reconnects, form exposure, or concurrent attempt tokens. They should trigger review, not automatic accusations. Store derived integrity events with limited retention. Provide a documented appeal and reinstatement process.

8.5 Item quality

Pilot each item where possible. Track difficulty, discrimination, distractor selection, omission, time, and differential performance across sufficiently large privacy-safe groups. Retire ambiguous, exposed, or non-discriminating items. Never make high-stakes decisions from small demographic samples. A reviewer must approve item changes and document the rationale.

9. Certification and credential lifecycle

9.1 Eligibility and issuance

Certificate eligibility is a server-evaluated policy tied to a published course version:

- learner has an active enrollment in the required path;
- all required activities and labs are complete;
- final exam status is passed;
- any required identity or manual review is approved;
- no integrity hold or revocation block exists;
- learner accepted the certificate display-name policy.

Issuance occurs exactly once per learner, certification version, and qualifying attempt. A transactional RPC locks the attempt, validates eligibility, allocates a non-sequential public verification code, creates the credential record, writes an audit event, and queues PDF generation. Retrying returns the existing credential.

9.2 Credential contents

The downloadable certificate should show:

- VeriTrust issuer identity and brand;
- learner-approved display name;
- certification title and level;
- issue and expiry dates;
- credential ID and public verification URL;
- QR code resolving to the verification URL;
- certification version or standard version;
- authorized signatory label;
- statement of demonstrated outcomes without overstating forensic authority.

The PDF should include metadata and a document hash stored on the credential. If stronger portability is later needed, add an Open Badges 3.0 or W3C Verifiable Credentials representation after a dedicated standards and privacy review. Do not block the MVP on decentralized credentials.

9.3 Public verification

GET /api/learning/certificates/verify/:publicCode is public, aggressively rate-limited, and returns only:

- status: valid, expired, revoked, or not found;
- display name according to learner consent policy;
- certification name, version, issuer, issue date, expiry date;
- credential ID and outcome summary;
- revocation date and broad reason category if revoked.

It must not expose email, user ID, organization membership, exam score, answers, attempt history, or internal audit data. Public codes need at least 128 bits of randomness. Verification pages use `noindex` by default unless the learner explicitly enables public profile discovery.

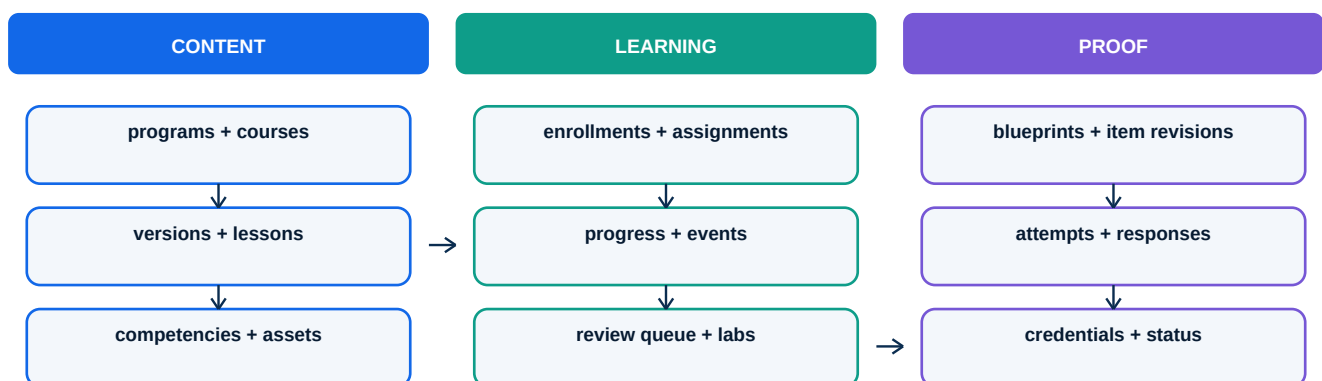
9.4 Expiry, renewal, and revocation

Recommended foundation validity is two years. Renewal can require a shorter delta course and renewal assessment if the competency standard remains compatible; major standard changes require the full current assessment.

Revocation states: active -> suspended -> active or revoked. Reasons include issued in error, confirmed integrity violation, identity dispute, or policy breach. Only authorized issuer administrators can revoke. Every action requires reason, evidence reference, actor, timestamp, and audit event. The learner is notified and can appeal. Revoked PDFs remain cryptographically identifiable but verification returns revoked.

10. Data model and database constraints

Figure 4. Versioned learning data boundaries



Published content, assessment inputs and credentials remain reconstructable after future updates.

All tenant-owned tables include `org_id` where organization scope is meaningful, `created_at`, `updated_at`, and audit-safe actor fields. Use UUID primary keys internally and random public codes externally. Published versions are immutable.

Domain	Core tables	Important constraints
Catalog	<code>learning_programs</code> , <code>learning_courses</code> , <code>course_versions</code> , <code>course_modules</code> , <code>course_lessons</code> , <code>lesson_blocks</code>	Unique slugs; one current published version; immutable published rows; ordered child uniqueness
Competency	<code>competencies</code> , <code>course_competencies</code> , <code>activity_competencies</code>	Stable competency code and version; many-to-many objective mapping
Enrollment	<code>learning_enrollments</code> , <code>learning_assignments</code> , <code>cohorts</code> , <code>cohort_members</code>	One active enrollment per learner/course version; source and due-date history
Progress	<code>lesson_progress</code> , <code>activity_progress</code> , <code>learning_bookmarks</code> , <code>learning_event_receipts</code>	Unique learner/activity/version; monotonic completion; idempotent event receipt

Domain	Core tables	Important constraints
Assessment	<code>assessments</code> , <code>assessment_versions</code> , <code>exam_blueprints</code> , <code>question_items</code> , <code>question_revisions</code> , <code>answer_options</code>	No correct-answer reads by learner role; immutable published revisions
Attempts	<code>assessment_attempts</code> , <code>attempt_items</code> , <code>attempt_responses</code> , <code>attempt_events</code> , <code>attempt_reviews</code>	Legal state transitions; unique item instances; server-owned timer; append-only events
Credentials	<code>certification_definitions</code> , <code>certification_versions</code> , <code>credentials</code> , <code>credential_artifacts</code> , <code>credential_status_events</code>	Unique issuance key; random public code; append-only status history; document hash
Content ops	<code>content_reviews</code> , <code>content_publications</code> , <code>content_assets</code>	Approval separation; manifest hash; asset ownership and retention
Engagement	<code>learning_preferences</code> , <code>review_queue</code> , <code>notification_preferences</code> , <code>learning_experiments</code>	Explicit consent and frequency limits; experiment assignment stability
Audit	<code>learning_audit_events</code> , <code>learning_security_events</code>	Append-only, actor and correlation IDs, protected metadata, retention class

Key transaction RPCs:

- `publish_course_version(version_id)` validates approvals, objectives, required activities, assets, and manifest hash.
- `enroll_in_course(course_version_id, source, assignment_id)` enforces entitlement and prerequisites.
- `record_learning_event(idempotency_key, activity_id, event_type, payload)` stores the receipt and updates progress atomically.
- `start_assessment_attempt(assessment_version_id)` enforces eligibility and limits, then materializes the randomized form.
- `save_attempt_response(attempt_id, attempt_item_id, revision, response)` uses optimistic concurrency and audit events.
- `submit_and_score_attempt(attempt_id)` closes the attempt, scores from server-only keys, applies objective floors, and records the result.
- `issue_credential(qualifying_attempt_id)` locks eligibility and creates one credential.
- `set_credential_status(credential_id, status, reason)` enforces issuer permission and writes history.

RLS principles:

- public can read only catalog preview views and minimal credential verification views;
- learners read published content they are entitled to, their own progress, attempts without answer keys, and their own credentials;
- instructors read assigned cohort progress and review artifacts, never unrelated organizations;
- organization admins read aggregate and assignment data in their organization, with score detail governed by policy;
- only service-role functions can publish versions, score certification attempts, issue credentials, or change credential status;
- authoring tables require explicit content roles; publication requires a different approval capability.

11. API design and Vercel integration

Add one Vercel function, `api/learning.js`, with route handlers under `lib/routes/learning/`. This moves the function count from six to seven, preserving headroom under the repository's 12-function budget.

11.1 Learner and public routes

Method and route	Purpose	Auth
GET /api/learning/catalog	Filtered catalog and recommendations	Optional
GET /api/learning/courses/:slug	Preview or entitled published course manifest	Optional / required for full content
POST /api/learning/enrollments	Enroll in a course version	Required
GET /api/learning/me	My Learning summary and next action	Required
GET /api/learning/enrollments/:id	Enrollment, outline, objective progress	Required
POST /api/learning/events	Record idempotent activity event	Required
GET /api/learning/review-queue	Due spaced-practice activities	Required
POST /api/learning/assessments/:id/attempts	Start eligible attempt	Required
GET /api/learning/attempts/:id	Current form, status, server time	Required
PUT /api/learning/attempts/:id/responses/:itemId	Save response with revision	Required
POST /api/learning/attempts/:id/submit	Transactional final submit and score	Required
GET /api/learning/certificates	Owner's credentials	Required
GET /api/learning/certificates/verify/:code	Minimal public verification	Public, rate-limited
GET /api/learning/certificates/:id/download	Short-lived signed PDF download	Owner or authorized admin

11.2 Organization and content routes

- assignments: create, list, update deadline, cancel;
- cohorts: create, add/remove members, aggregate progress;
- reports: completion, objective mastery, expiry, and export jobs;
- content drafts: create course version, modules, lessons, blocks, and assets;
- review: submit, approve, reject, accessibility sign-off, publish;
- credentials: suspend, reinstate, revoke, and resolve appeal.

Admin mutations require role and capability checks in both the handler and RLS/RPC. CSV exports should be asynchronous for large organizations and placed in a private storage bucket with short-lived signed URLs.

11.3 API rules

- JSON envelopes use { `ok`, `data`, `error`, `meta` } consistently.
- Mutation requests require `Idempotency-Key` or a client-generated event ID.
- Cursor pagination replaces large offsets for events, catalog search, attempts, and reports.
- ETags and version fields protect course manifests and response updates.
- Error codes are stable and safe: `AUTH_REQUIRED`, `NOT_ENTITLED`, `PREREQUISITE_INCOMPLETE`, `ATTEMPT_LIMIT`, `ATTEMPT_EXPIRED`, `VERSION_CONFLICT`, `INTEGRITY_REVIEW`, and `CERTIFICATE_NOT_AVAILABLE`.
- Client-facing errors never include SQL, provider responses, scoring keys, or private policy details.

- Rate limits separate catalog reads, progress writes, assessment writes, public verification, and admin exports.
- Correlation IDs flow from the browser through API logs, database audit events, and background jobs.

12. Frontend component and state plan

Add `assets/js/learning.js`, `assets/js/assessment.js`, `assets/js/certificate.js`, and `assets/css/learning.css`. Keep `config.js`, `supabase-client.js`, `site.js`, and the existing design tokens. Extend the Supabase client wrapper with typed learning API methods rather than scattering raw `fetch` calls across pages.

Reusable UI components:

- course card, program card, progress ring, objective bar, due-date pill;
- course outline with locked, available, in-progress, and completed states;
- lesson block renderer with a registry by block type and schema version;
- activity feedback panel and retry state;
- sticky Continue action driven by server next-action data;
- attempt timer synchronized to server time;
- autosave status with saved, pending, retrying, conflict, and offline states;
- certificate card and verification status panel;
- skeleton, empty, expired-session, entitlement, and error states;
- organization assignment and cohort progress tables.

Use a small state store per page, not global mutable DOM state. The server response is authoritative for entitlement, completion, attempts, scores, and credentials. Browser storage can cache published manifests and pending low-stakes events; it must not contain answer keys, certificate signing secrets, service credentials, or authoritative exam timers.

Route access matrix:

Route class	Signed out	Learner	Org admin	Content role
Catalog preview	View	View	View	View
Course preview	View	View	View	View
Lesson content	Redirect to auth	Entitled only	Entitled only	Preview draft if assigned
Assessment	Redirect to auth	Eligible own attempt	Own attempt only	Preview sandbox only
My Learning	Redirect to auth	Own	Own plus admin link	Own
Learning admin	Redirect to auth	Denied	Organization scope	Scope by capability
Certificate verify	Minimal public view	Minimal public view	Minimal public view	Minimal public view

13. Product labs and VeriTrust integration

Labs are the strongest connection between learning and the existing platform. They should use a dedicated lab mode rather than asking users to upload private real-world content.

Recommended lab pattern:

1. The learning API issues a short-lived lab session for a seeded scenario.
2. The lesson launches a detection or gateway page with `lab_session` and scenario identifiers.
3. Detection inputs are pre-approved synthetic fixtures or safe strings stored with the course version.
4. Existing scan APIs run normally but tag the scan as training and exclude it from billable production usage if the plan policy allows.
5. The lab validator checks required actions and reasoning, not whether an AI model happened to return one exact score.
6. The learning service records completion evidence and returns the learner to the canonical next activity.

Training scans must be visually labeled and separable from operational scan history. They must not contaminate production analytics, model evaluation, billing usage, or security incident records. A learner can optionally repeat the workflow with personal content after leaving the lab, under the normal product privacy and billing rules.

14. Entitlements, commerce, and organization features

Extend `PLAN_DEFAULTS`, plan rows, and billing snapshots with explicit learning features:

- `learning_catalog_access`
- `learning_certificate_access`
- `learning_admin_access`
- `learning_seat_limit`
- `learning_assignment_limit`
- `learning_export_access`
- `learning_proctored_exam_access` for future use

The foundation course can be included in Free to drive product competence, while advanced paths, certificates, organization assignments, and analytics can be Pro or enterprise features. Do not paywall required safety information or a learner's already-earned verification record. If a subscription ends, learners keep access to their credential verification and previously downloaded certificate; active premium content follows the documented grace policy.

Seat assignment must be explicit and auditable. Organization admins invite or assign existing members, see consumed and available seats, and can release a seat subject to retention policy. Never automatically expose a learner's detailed wrong answers to an employer. Default organization reporting should show enrollment, required completion, certification status, expiry, and objective-level aggregate risk where permitted.

15. Security, privacy, and abuse prevention

Threats include answer-key extraction, unauthorized content access, attempt replay, response tampering, timer manipulation, credential forgery, public-code enumeration, cross-tenant reporting, instructor privilege escalation, malicious course HTML, unsafe asset upload, notification abuse, and analytics over-collection.

Required controls:

- render lesson blocks from a strict schema; sanitize any authored HTML and prohibit arbitrary scripts;
- private content assets use signed short-lived URLs and MIME validation;
- server-only answer keys and scoring logic;
- random attempt tokens bound to user, assessment version, and expiry;
- optimistic response revisions and append-only attempt events;
- transactional attempt submit and credential issue;
- 128-bit or stronger public verification codes; rate limits and generic not-found responses;
- row-level organization isolation plus handler capability checks;
- separation of author, reviewer, publisher, and credential administrator permissions;
- audit sensitive admin reads and every content publication, score override, credential action, and export;
- encrypt provider secrets and keep all certificate signing material server-side;
- content security policy, CSRF-resistant bearer use, strict CORS, input size limits, and safe error envelopes;
- anti-automation controls triggered by risk, not blanket friction;
- dependency, secret, SAST, and migration scanning in CI;
- documented incident response for exposed questions, scoring defects, and incorrect certificates.

Privacy decisions:

- define retention separately for learning events, assessment answers, integrity signals, certificates, and audit logs;
- allow account deletion while preserving a legally necessary minimal credential revocation record, if policy requires;
- provide download and correction workflows for profile and credential display name;
- do not collect camera, microphone, screen, or biometric data in the standard tier;
- require a data-protection review before proctoring or identity-vendor integration;
- keep free-text reflections private by default and exclude them from analytics content capture;

- document organization visibility before a learner accepts an assignment.

16. Accessibility, localization, and inclusive assessment

Target WCAG 2.2 AA. Acceptance includes:

- complete keyboard operation and visible focus;
- semantic headings, landmarks, labels, status announcements, and error summaries;
- no interaction that depends only on drag, hover, color, sound, or time pressure;
- captions, transcripts, audio description where needed, and text alternatives;
- reflow at 320 CSS pixels and 200 percent zoom without lost content;
- reduced-motion support and no flashing hazards;
- accessible timer warnings and approved extra-time accommodations;
- equivalent non-drag alternatives for sorting, matching, and hotspot blocks;
- screen-reader tested lesson, quiz, exam, and certificate verification flows;
- plain-language feedback and readable code/URL examples.

Store all learner-facing content through localization keys or versioned localized content rows. A course version can declare required and available locales. Translation must not silently reuse the source assessment: translated items need linguistic and subject review, and difficulty data should be monitored separately. Dates, durations, names, reading direction, and number formats must follow locale.

17. Analytics, experimentation, and reporting

Use a first-party, privacy-minimizing event vocabulary inspired by learning record statements. Events should describe meaningful actions:

- catalog_viewed, recommendation_opened, enrollment_created;
- lesson_started, block_completed, knowledge_check_submitted, feedback_viewed;
- hint_opened, remediation_started, lab_started, lab_completed;
- practice_attempt_started/submitted, certification_attempt_started/submitted;
- certificate_issued, certificate_viewed, certificate_verified;
- reminder_opted_in, reminder_sent, reminder_opened, reminder_disabled.

Each event includes event ID, user ID or approved anonymous ID, organization ID where applicable, session ID, course and content versions, activity ID, timestamp, client version, experiment assignment, and a bounded metadata schema. Do not place full lesson text, learner answers, URLs from personal scans, email content, access tokens, or unnecessary device fingerprints into analytics.

Dashboards:

- Product: discovery funnel, activation, module drop-off, completion, certificate, return, and satisfaction.
- Learning: objective mastery, common misconceptions, practice-to-exam improvement, item quality, and remediation effectiveness.
- Organization: assigned, started, completed, certified, expiring, and overdue by cohort.
- Operations: API errors, event-sync lag, exam autosave latency, scoring queue, PDF generation, and verification availability.
- Equity and accessibility: outcome gaps using minimum group-size thresholds and approved demographic data only.

Experiment lifecycle: hypothesis -> metric and guardrails -> eligibility and sample plan -> stable assignment -> run -> analyze -> document -> ship or remove. Avoid testing high-stakes scoring thresholds casually. Assessment-policy changes require governance and versioning, not normal growth experimentation.

18. Reliability, performance, and observability

Service objectives for launch:

- catalog and course manifest API: 99.9 percent monthly availability, p95 under 500 ms when cached;
- authenticated My Learning: 99.9 percent, p95 under 800 ms;
- progress event acknowledgement: 99.9 percent, p95 under 700 ms;
- exam response save: 99.95 percent during active attempts, p95 under 600 ms;
- certificate verification: 99.95 percent, p95 under 400 ms;
- certificate PDF generation: 99 percent completed within 60 seconds, retryable without duplicate issuance.

Cache public catalog responses at the edge with versioned ETags. Cache immutable published course manifests and assets aggressively. Do not cache personalized progress or exam forms in shared caches. Use cursor pagination and precomputed organization aggregates for scale.

Background jobs should handle certificate PDF rendering, reminders, large exports, review-queue scheduling, content indexing, and analytics rollups. Reuse the gateway worker's signed dispatch, bounded batches, retries, dead-letter handling, heartbeats, and recovery patterns where practical, but use separate job types and retention policies.

Observability requirements:

- structured logs with correlation, organization, route, status, latency, and safe error code;
- metrics for request count, failure, latency, authorization denial, idempotency replay, job age, attempt save conflict, and credential issuance;
- traces across API, RPC, storage, and worker for high-value flows;
- alerts on verification outage, elevated attempt-save failures, scoring backlog, duplicate-issuance constraint errors, cross-tenant denial anomalies, and publication failure;
- runbooks for auth outage, Supabase degradation, certificate generation failure, exposed question bank, bad content release, and incorrect scoring.

19. Testing and quality strategy

19.1 Automated layers

Layer	Required coverage
Static checks	JavaScript syntax, HTML navigation, route rewrite targets, function budget, content schemas, forbidden placeholder links
Unit	progress calculations, next-action resolver, prerequisite graph, blueprint selection, scoring, objective floors, expiry, public code generation
Database	constraints, RLS allow/deny matrix, transaction idempotency, concurrent response writes, duplicate issuance, legal state transitions
API contract	auth, roles, validation, errors, pagination, ETag/version conflict, rate limits, redaction
Component	block renderers, feedback, offline state, timer, progress, certificate status
End-to-end	discover -> sign in -> enroll -> learn -> lab -> practice -> exam -> issue -> verify
Accessibility	automated axe-style rules plus manual keyboard and screen-reader scripts
Security	authorization fuzzing, tenant isolation, XSS content payloads, answer-key leakage, code enumeration, replay, upload validation
Performance	catalog load, large course outline, autosave burst, attempt submit, organization report, verification spike
Recovery	network loss, token refresh, stale version, worker retry, partial PDF failure, interrupted attempt

19.2 Certification release gates

No certification can launch until:

- subject-matter, assessment, legal/claims, privacy, and accessibility sign-offs are recorded;
- every objective has adequate instruction, practice, and scored coverage;
- answer keys are absent from learner APIs and browser bundles;
- scoring recomputation is deterministic from stored attempt inputs;
- duplicate issuance and public-code enumeration tests pass;
- cut-score pilot and item review are documented;
- accommodations and appeal workflows are operational;
- certificate verification and revocation are tested during provider and worker failures;
- support, incident, and rollback runbooks are approved.

20. Delivery roadmap, dependencies, and team shape

Figure 5. Controlled 14 to 18 week delivery sequence



Foundations and progress reliability precede assessment, credentials and engagement optimization.

Assuming a focused cross-functional squad, target a 14 to 18 week foundation release. Calendar estimates must be revised after schema-baseline discovery and content inventory.

Phase	Indicative duration	Engineering outcome	Product/content outcome	Exit gate
0. Baseline and contracts	1-2 weeks	Track migrations; access matrix; API/error/event conventions; navigation config	Final learner personas, launch outcomes, certification claims	Architecture and governance approval
1. Catalog and enrollment	2 weeks	Learning function, catalog schema/API, page shells, entitlement hooks	Course outline, objectives, preview content	Public/auth flows and RLS pass
2. Lesson and progress	3 weeks	Block renderer, progress RPC, next action, offline retry	First three production modules and accessibility assets	Progress consistency and accessibility pass
3. Labs and My Learning	2-3 weeks	Lab sessions, training scan tags, dashboard integration, review queue	All guided labs and remediation	No production billing/analytics contamination
4. Assessment engine	3 weeks	Blueprint, attempt state machine, response save, server scoring, review	Item bank, practice exam, exam forms	Integrity and deterministic scoring gates
5. Credentials and admin	2-3 weeks	Issuance RPC, PDF worker, verify route, assignments, cohort reports	Certificate policy, template, expiry, appeals	Verification, revocation, privacy gates
6. Pilot and hardening	2 weeks	Load, recovery, security fixes, monitoring, runbooks	Pilot, item analysis, cut-score review, content corrections	Go/no-go review

Phase	Indicative duration	Engineering outcome	Product/content outcome	Exit gate
7. General release	Ongoing	Gradual rollout, SLO operation, backlog	New cohorts, reinforcement, content cadence	KPI and incident review

Minimum roles: product owner, tech lead/backend engineer, frontend engineer, product designer, learning designer, security subject-matter expert, assessment specialist, QA/accessibility owner, and part-time privacy/legal and operations support. One person may cover multiple roles, but author, reviewer, publisher, and credential-revocation permissions should remain separated in production.

External or policy dependencies:

- deployed Supabase schema baseline and migration ownership;
- plan and pricing decision for certificates and organization seats;
- certification name, claims, validity, cut-score, retake, appeal, and revocation policies;
- content and media production capacity;
- email/reminder provider only if reminders are included at launch;
- certificate signing and PDF rendering method;
- privacy and terms updates;
- support ownership and incident escalation.

21. Prioritized implementation backlog

P0 - required for a trustworthy launch

- Establish tracked Supabase migrations and RLS test harness.
- Centralize global and workspace navigation; add Learn routes and nav integrity contracts.
- Implement catalog, published version, enrollment, progress, and competency schemas.
- Implement learning route dispatcher and auth/entitlement middleware.
- Build accessible catalog, My Learning, course, and lesson shells.
- Implement event idempotency, next-action resolution, and offline retry.
- Build question bank, blueprint, attempt state machine, response revisions, and server scoring.
- Implement certification eligibility, exactly-once issuance, PDF generation, public verification, expiry, and revocation.
- Add audit logs, rate limits, cross-tenant tests, answer-key leakage tests, and operational alerts.
- Produce and review the complete foundation course, practice set, final item bank, accommodations, and appeals.

P1 - high-value engagement and organization readiness

- Spaced review queue and adaptive remediation.
- Organization assignments, cohorts, deadlines, and aggregate reporting.
- First-party reminder preferences and scheduled reminders.
- Training-mode integrations with phishing, link, deepfake, and gateway workflows.
- Content author/review/publish interface or controlled internal authoring scripts.
- Certificate sharing controls and public profile opt-in.
- Localization framework and first translated course after validation.

P2 - scale and advanced credentials

- Advanced paths, stackable badges, continuing education, and renewal deltas.
- Standards-based Open Badges or Verifiable Credential export.
- Enterprise SSO/SCIM and LMS interoperability such as LTI or xAPI export if demanded.
- Proctored or verified-identity tier after privacy and accessibility review.
- Instructor-led cohorts, moderated discussion, and rubric-based submissions.
- Recommendation models only after sufficient consented data and baseline rules.

22. Definition of done and launch scorecard

The learning and certification module is production-ready only when the following are true:

Functional

- A signed-out visitor can understand the course and return to the intended route after authentication.
- An entitled learner can enroll, resume across devices, complete accessible activities, recover from network failure, and see accurate progress.
- The next-action resolver never sends the learner to completed, locked, unpublished, or unauthorized content.
- A learner can practice, start one eligible certification attempt, autosave responses, submit exactly once, receive a deterministic result, and access remediation.
- A qualifying pass issues one credential; verification, PDF download, expiry, suspension, reinstatement, revocation, and appeal states work.
- Organization admins can assign learning and see only authorized organization data.

Security and integrity

- Published content and assessment versions are reconstructable and immutable.
- No answer key, signing secret, service key, private learner answer, or cross-tenant data appears in client bundles, logs, analytics, or public verification.
- Mutation replay is safe; public codes resist enumeration; scoring and issuance are transactional.
- Authorization, RLS, rate-limit, upload, XSS, and abuse tests pass.

Experience and inclusion

- Core flows pass WCAG 2.2 AA automated and manual checks.
- Desktop and mobile flows have no dead end, placeholder link, inaccessible control, or ambiguous next action.
- Loading, empty, entitlement, expired session, offline, conflict, interrupted attempt, and revoked credential states are designed and tested.
- Learners can control reminders, display name, sharing, and relevant privacy settings.

Operations and outcomes

- SLO dashboards, alerts, logs, runbooks, backups, migration rollback, and support ownership are ready.
- Pilot evidence supports the exam policy and removes defective items.
- Activation, completion, mastery, certification, accessibility, satisfaction, and integrity guardrails are measured from day one.
- General release uses a feature flag and staged rollout, with a tested path to disable enrollment or new attempts without invalidating existing progress and credentials.

The strongest implementation sequence is: versioned data and authorization first, lesson/progress reliability second, assessments and credentials third, engagement optimization fourth. This preserves trust while still creating an experience that is clear, practical, motivating, and deeply integrated with VeriTrust.

Recommended delivery principle: establish versioned data, authorization and reliable progress first; then add controlled assessment and credentials; optimize engagement only against measured learning and accessibility outcomes.